Vulnerability Exploited: CVE-2025-55177 (Social Engineering–Based Exploitation)

Mahrukh

---

1. Introduction:

Aim here is building a secure digital setup where one specific weakness gets tested in practice. This part puts stress on doing things yourself so it becomes clear how hackers might misuse flaws. Seeing the damage firsthand shows exactly what happens when systems fall victim. Learning comes from acting, not just reading - experience reveals the consequences.

Aiming to boost hands-on security knowledge, this effort involves building both attacking and target systems. Starting with setup, one machine acts while the other responds under controlled conditions. Execution follows, where simulated breaches reveal how vulnerabilities can be actively used. Outcomes show what happens when defenses are weak or missing. Attention shifts to configurations - how systems are arranged makes a difference. People play a role too, since choices affect safety just as much as settings do. Lessons point toward steps that lower danger before harm occurs. Real incidents mirror these tests, making practice meaningful.

---

2. Description:

2.1 Virtual Machines and Their Functions

A digital setup ran on two separate systems built through emulation tools. One box, running Kali Linux, played the role of the one launching attacks. The second, set up with Ubuntu Linux, served as the receiving end where harmful code landed. From the Kali side came the creation and delivery of dangerous scripts. On the flip side, the Ubuntu instance responded to those inputs as if it were a real user scenario.

2.2 Network Configuration

A single private network linked both virtual machines. Communication stayed within that boundary, kept separate from outside systems. Safety during testing relied on this kind of separation. Experiments involving vulnerabilities needed this controlled space.

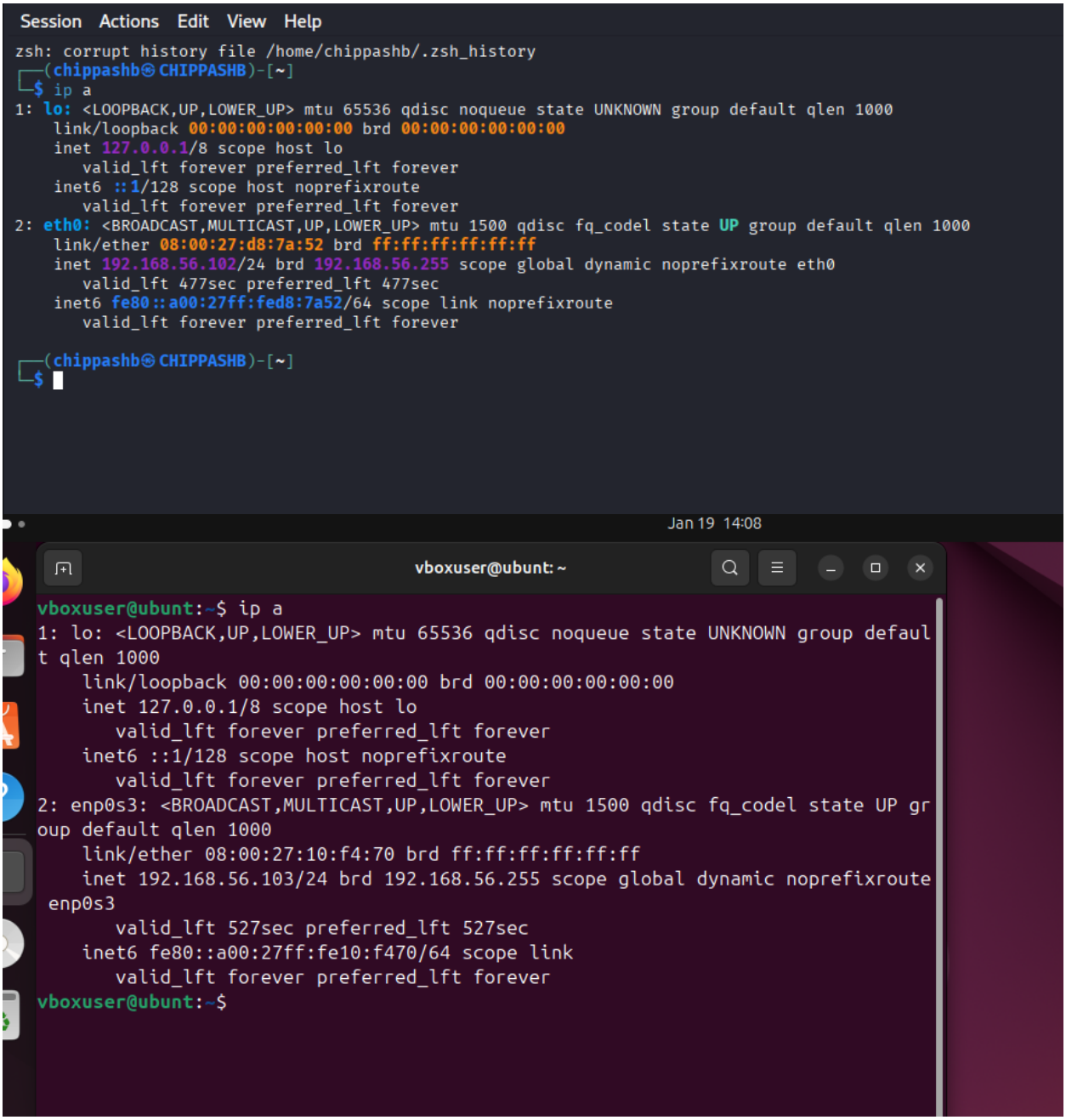
A test came first checking if the two systems could talk to each other across the network. Only after seeing a live connection did anything else move forward. Signals passed back and forth without delay, showing they lived on the same segment. Communication worked, so the next phase began.

---

3. Step-by-Step Exploitation Process

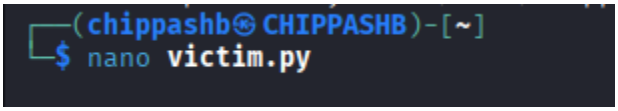
3.1 Network Check and IP Find

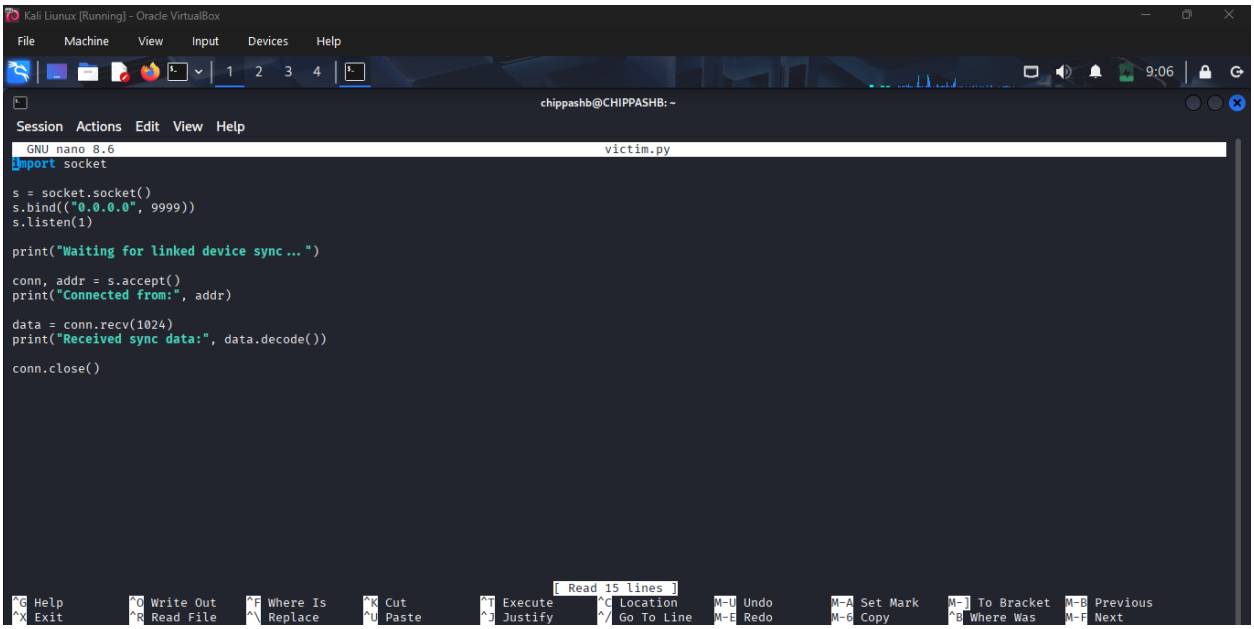
Out of the blue, checking if the attacker and target computers could talk started things off. From each machine, pings went out - just to see what would answer back. When replies came through without issues, it meant the setup worked as needed.



3.2 Writing Scripts with Python

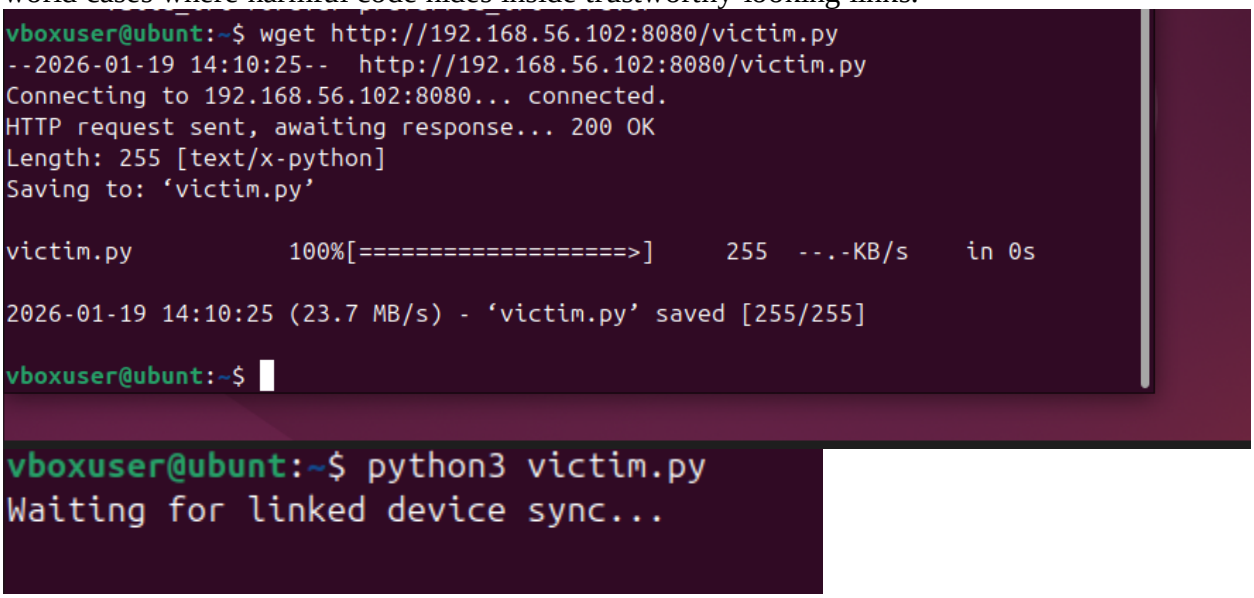
A file took shape inside Nano, typed out line by line on
the attacking system. Running it checked whether messages moved correctly between systems. E
ach command built a starting point, not an endpoint. Making sure code runs at all matters more t
han what comes after.





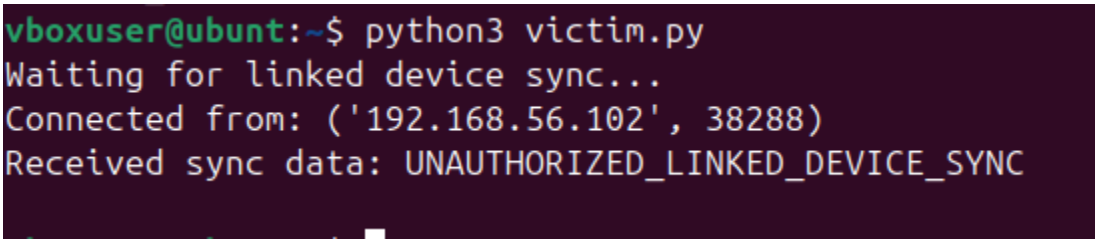
3.3 Payload Delivered to Victim Device

A script, once ready, moved to the target computer via a small HTTP server set up on the hacker's device. From there, the infected machine pulled down the file, acting like real-world cases where harmful code hides inside trustworthy-looking links.



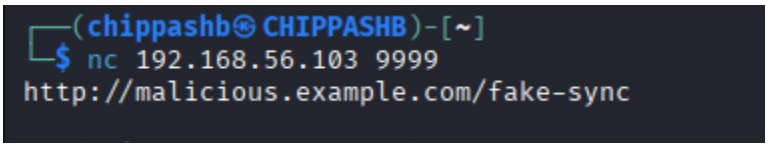
3.4 Fake Message Execution

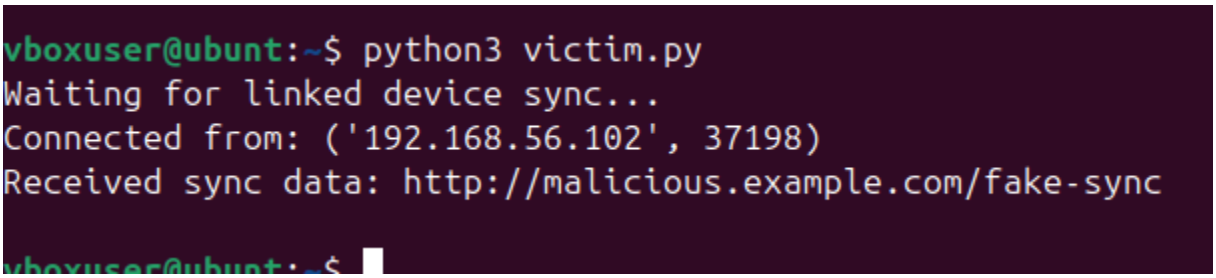
A signal went out from the attacker device after the script arrived, pretending to be normal traffic. Right then, it became clear if the target machine would react to harmful data. That moment showed exactly what hackers check before moving forward - whether a system opens the door without suspicion.



3.5 Fake URL Exploitation

A sneaky link got sent to target device, stretching out the exploit window. That move copied a re al-world phising attempt, showing just how easily it can be nudged into clicking harmful web address.





---

4. Impact Review and Risk Reduction

4.1 Impact Analysis

A breach like this might let attackers sneak in, run harmful code, steal information, then dig deep er into systems. Tricking people through manipulation poses serious risks since it uses faith in ot hers instead of broken software - this kind of trick often slips past standard defenses.

4.2 Mitigation Strategies

A fresh start helps break old habits.
Users who learn about online risks every few months tend to stay alert. Staying away from unfa miliar links makes a difference over time. Filters on emails and websites catch problems early. Systems patched often run more smoothly than those left behind. Watching network traffic show s odd behavior before it spreads.

---

5. Conclusion:

A fake scenario showed how people can be tricked into giving up access. From this test came cle ar examples of psychological tricks used in digital attacks. Awareness training turned out to matt er just as much as strong passwords. Settings that block risky actions helped reduce success by at tackers. Real habits - not perfect tools - decided whether defenses held.